

第三章 关系数据库标准语言 SQL —— AI 讲解

根本矛盾

数据库里存的是一张张「冰冷的二维表」，而我们想用「人话」去问它问题、改它数据、藏它细节。SQL 就是那个翻译官 + 记事本 + 守门员：既能盖房子（DDL）、又能查数据（SELECT）、还能改数据（DML）、还能把大表包成小窗（视图）。

一、数据定义（DDL）—— 给数据库「盖房子、编目录」

SQL 的 DDL（Data Definition Language）干三件事：建模式 → 建表 → 建索引。你把它想象成：

概念	类比	关键字
模式 (SCHEMA)	楼栋（容纳多个表的命名空间）	CREATE SCHEMA / DROP SCHEMA
基本表 (TABLE)	房间（真实存数据的地方）	CREATE TABLE / ALTER TABLE / DROP TABLE
索引 (INDEX)	房间门口的小书签（加速查找）	CREATE INDEX / ALTER INDEX / DROP INDEX

恍然大悟点：模式是命名空间，不是「数据库」。一个用户可以拥有多个模式，每个模式里又有多个表。所以「数据库 > 模式 > 表 > 行/列」是四级关系。

1. CREATE SCHEMA

```
-- 为用户 WANG 定义一个 S-T 模式
CREATE SCHEMA "S-T" AUTHORIZATION WANG;

-- 省略模式名时，隐式使用用户名作为模式名
CREATE SCHEMA AUTHORIZATION WANG;
```

2. CREATE TABLE

```
CREATE TABLE TAB1 (
  Sno VARCHAR(10),
  Cno NUMBER(10),
  Grade INT NOT NULL,
  PRIMARY KEY (Sno, Cno),           -- 表级：组合主码
  FOREIGN KEY (Sno) REFERENCES TAB2(Sno) -- 表级：外码
);
```

数据类型速记表

类型	含义	备注
CHAR(n)	定长字符串	不够补空格
VARCHAR(n)	变长字符串	节省空间
INT / SMALLINT / BIGINT	4B / 2B / 8B 整数	视范围选



FLOAT(n)	精度 $\geq n$ 位的浮点	
NUMBER(n)	长度为 n 的数字	Oracle 风格
DATE	YYYY-MM-DD	
TIME	HH:MM:SS	

完整性约束：列级 vs 表级

约束	列级（直接跟列写）	表级（写在最后）
PRIMARY KEY	单列主码时可直接写列后	多列组合主码必须用表级
NOT NULL	可	不可（NOT NULL 只能列级）
UNIQUE	可	可
CHECK(条件)	可	可
FOREIGN KEY ... REFERENCES	不可	必须表级

金句：列级约束给「单兵」用，表级约束给「联合部队」用——组合主码和外码都是「联合部队」。

2.5 在模式中定义表的三种方式

一个模式包含多个基本表，DDL 中有三种方式把表挂到模式上：

```
-- (1) 创建表时显式指出模式（表名前缀 模式名）
CREATE TABLE S-T.TAB1 (
  Sno VARCHAR(10) PRIMARY KEY,
  Sname VARCHAR(20)
);

-- (2) 创建模式时直接定义表（模式定义里嵌表定义）
CREATE SCHEMA S-T AUTHORIZATION WANG
CREATE TABLE TAB1 (
  Sno VARCHAR(10) PRIMARY KEY,
  Sname VARCHAR(20)
);

-- (3) 设置当前所属模式（之后建表不用再带模式名）
SET SCHEMA 'S-T';
CREATE TABLE TAB1 (...);
```

金句：三种方式的本质区别是「什么时候把表挂到模式上」——显式前缀是边建边挂、CREATE SCHEMA 是先建楼再装修、SET SCHEMA 是先调航向再下针。

3. ALTER TABLE 六种姿势



```

-- (1) 加列
ALTER TABLE SC ADD COLUMN Time DATE;

-- (2) 加列级约束
ALTER TABLE SC ADD UNIQUE(Cname);

-- (3) 加表级约束
ALTER TABLE SC ADD FOREIGN KEY(Cno) REFERENCES Student(Cno);

-- (4) 删列 (CASCADE 一起删引用该列的视图)
ALTER TABLE SC DROP Sage CASCADE;

-- (5) 删约束
ALTER TABLE SC DROP CONSTRAINT 约束名 CASCADE;

-- (6) 改列类型
ALTER TABLE SC ALTER COLUMN Sage INT;

```

4. CASCADE vs RESTRICT —— 删东西的两种态度

CASCADE = 「砍树枝」：被删对象的依赖（外码引用、视图）一起删。

RESTRICT = 「金钟罩」：只要还有别的东西依赖它，拒绝删除。

这条规则在 DROP SCHEMA / DROP TABLE / DROP COLUMN / DROP CONSTRAINT / DROP VIEW 里反复出现，是考试重点。

5. 索引

```

CREATE UNIQUE INDEX SCno ON SC(Sno ASC, Cno DESC); -- 唯一索引
CREATE CLUSTER INDEX ... ON ...; -- 聚簇索引（物理顺序）
ALTER INDEX SCno RENAME TO SCSno; -- 改名
DROP INDEX SCSno; -- 删除

```

恍然大悟点：UNIQUE 索引 ≠ UNIQUE 约束。UNIQUE 索引是物理加速 + 唯一；UNIQUE 约束是逻辑要求。一个 UNIQUE 约束背后会自动建一个 UNIQUE 索引。

二、SELECT 语句骨架 —— 五个「子句盒子」

```

SELECT [DISTINCT|ALL] 目标列表表达式  ← 输出什么
FROM 表名/视图名                ← 从哪取
WHERE 条件表达式                  ← 行过滤
GROUP BY 列名 HAVING 条件         ← 分组 + 组过滤
ORDER BY 列名 [ASC|DESC];        ← 排序

```

执行顺序（理解关键）：

FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY

金句：SELECT 不是「按行读出来的顺序」，SELECT 是「最后被执行的列投影」。先过滤、再分组、最后才决定显示哪些列。

三大「列投影」招式



```

SELECT X, Y          FROM TAB;      -- (1) 指定列
SELECT *             FROM TAB;      -- (2) 全部列
SELECT 2022-age AS birth FROM TAB;    -- (3) 计算 + 别名
SELECT DISTINCT X     FROM TAB;      -- (5) 去重 (DISTINCT 只能对整体去重)

```

三、WHERE 六种过滤条件 —— 「筛子家族」

类别	关键字	例句	陷阱
比较	= > < >= <= !=	WHERE X > 20	字符串要加单引号
范围	BETWEEN a AND b	WHERE age BETWEEN 20 AND 30	含端点
集合	IN (...) / NOT IN (...)	WHERE Sdept IN ('CS','MA')	等价于 = ANY
字符匹配	LIKE	WHERE Sname LIKE '刘_'	% 任意长, _ 单字符
空值	IS NULL / IS NOT NULL	WHERE Grade IS NULL	不能写 = NULL
复合	AND / OR	WHERE X>20 AND Y<10	AND 优先级 > OR

LIKE 通配符 + ESCAPE (每年必考)

```

-- 姓刘、全名 3 个汉字
WHERE Sname LIKE '刘_';

-- 课程名是 DB_Design (下滑线 _ 本身要转义)
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';

```

金句：一个汉字在 ASCII 里占 2 字节，所以 刘_ 是「姓刘 + 2 个汉字」= 共 3 个汉字。通配符是按字符数算的。

金句：= NULL 永远返回 UNKNOWN (既不是 TRUE 也不是 FALSE)，所以判断空值只能写 IS NULL。

四、聚集函数与 GROUP BY —— 从「明细」到「报表」

函数	作用	对空值
COUNT(*)	统计元组个数	空值不影响计数
COUNT(列)	统计非空值个数	跳过空值
AVG(列)	平均 (数值型)	跳过空值
SUM(列)	总和 (数值型)	跳过空值
MAX/MIN(列)	最大/最小	跳过空值

金句：聚集函数只认非空；只有 COUNT(*) 把「这条元组存在」当 1 个，连空值字段都算进去。

WHERE vs HAVING (最高频考点)

维度	WHERE	HAVING
----	-------	--------



作用对象	行（元组）	组（GROUP BY 出来的组）
时机	分组前	分组后
能否接聚集函数	不能	能
配 GROUP BY 用吗	不用也行	必须配 GROUP BY

```
-- 求 SC 表中，每门课的选课人数
SELECT Cno, COUNT(Sno)
FROM SC
GROUP BY Cno;

-- 求平均成绩 >= 90 的学生学号和平均成绩
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade) >= 90;
```

恍然大悟点：HAVING 里的 AVG(Grade) 是对每个组求一次；SELECT 里的 Sno 是分组的键。这两个搭配是「组内聚合 + 组键输出」的标准范式。

五、连接查询——多张表的「拼图游戏」

等值/非等值连接

```
-- 自然连接
SELECT Student.Sno, Sname, Ssex, Sage, Sdept, Cno, Grade
FROM Student, SC
WHERE Student.Sno = SC.Sno;
```

自身连接（同一张表当两张用）——必须取别名

```
SELECT FIRST.Cno, SECOND.Cpno
FROM Course FIRST, Course SECOND
WHERE FIRST.Cpno = SECOND.Cno;
```

金句：自身连接的精髓是「一个角色做两件事」——Course 表里既有「课程」又有「先修课」，用 FIRST/SECOND 两个别名把这两个身份拆开。

外连接——保留「孤儿元组」

```
-- 左外连接：以左表为主体，没匹配上的右表字段补 NULL
SELECT Student.Sno, Sname, Cno, Grade
FROM Student LEFT OUTER JOIN SC ON (Student.Sno = SC.Sno);
```

类型	保留哪边的孤儿
LEFT OUTER JOIN	左表（FROM 左侧）
RIGHT OUTER JOIN	右表
FULL OUTER JOIN	两边都保留



金句：等值连接是「严格匹配」——配不上的就被淘汰；外连接是「至少保留一边」——不上的用 NULL 撑场面。

多表连接（两两连接的链式调用）

```
-- 三表连接：Student + SC + Course
SELECT Student.Sno, Sname, Cname, Grade
FROM Student, SC, Course
WHERE Student.Sno = SC.Sno
      AND SC.Cno = Course.Cno;

-- 四表连接示例：加了 Teacher 表，查"选了某老师所授课程的学生"
SELECT DISTINCT Student.Sno, Sname, Cname, Tname
FROM Student, SC, Course, Teacher
WHERE Student.Sno = SC.Sno
      AND SC.Cno = Course.Cno
      AND Course.Tno = Teacher.Tno;
```

金句：多表连接 = 「两两连 + AND 串」。先连前两张，再把结果连第三张，依此类推。连接的字段一定是「外码 = 主码」或「同名列」。

六、嵌套查询 —— SQL 的「俄罗斯套娃」

嵌套查询 = 把一个 SELECT-FROM-WHERE 套到另一个的 WHERE 或 HAVING 子句里。

角色	别名	出现位置
外层（父查询）	上层	包裹者
内层（子查询）	下层	被包裹者

金句：子查询不能用 ORDER BY（ORDER BY 只能对最终结果排序，套娃里的娃娃没资格排）。

1. IN-子查询（不相关，最简单）

```
-- 找与刘晨同专业的同学（两步变一步）
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN
      (SELECT Sdept FROM Student WHERE Sname = '刘晨');
```

2. 比较运算符-子查询（必须返回**单个值**）

```
-- 找每个学生超过自己平均成绩的课程号
SELECT Sno, Cno
FROM SC X
WHERE Grade >= (SELECT AVG(Grade) FROM SC Y WHERE Y.Sno = X.Sno);
```

相关子查询 vs 不相关子查询：

类型	含义	执行方式
不相关子查询	子查询不依赖父查询	子查询算一次，结果给父查询用



相关子查询	子查询依赖父查询（用了父查询的列）	每取一个父元组，子查询重算一次
-------	-------------------	-----------------

3. ANY/ALL-子查询（与聚集函数的等价转换，必背）

谓词	语义	等价于
> ANY	大于子查询中某个值	> MIN
> ALL	大于子查询中所有值	> MAX
< ANY	小于子查询中某个值	< MAX
< ALL	小于子查询中所有值	< MIN
>= ANY	大于等于子查询中某个值	>= MIN
>= ALL	大于等于子查询中所有值	>= MAX
<= ANY	小于等于子查询中某个值	<= MAX
<= ALL	小于等于子查询中所有值	<= MIN
= ANY	等于子查询中某个值	IN
= ALL	等于子查询中所有值	几乎无意义
!= ANY	不等于子查询中某个值	
!= ALL	不等于子查询中任何值	NOT IN

口诀：ANY 找门槛最低的（MIN），ALL 找门槛最高的（MAX）。比「某个」松 → 用 MIN；比「所有」严 → 用 MAX。

```
-- 不是 CS 专业的学生中，比 CS 专业任意一个学生年纪小的学生
SELECT Sname, Sage
FROM Student
WHERE Sage < ANY (SELECT Sage FROM Student WHERE Sdept = 'CS')
AND Sdept <> 'CS';
```

4. EXISTS-子查询（双重否定套路，**最难的考点**）

EXISTS 只返回 TRUE/FALSE，不返回数据。EXISTS(子查询) = 子查询结果非空则为真。

套路一：「选修了 1 号学生所选全部课程」

命题转化：「对所有课程 y（1 号学生选 y → x 也选 y）」等价于「不存在这样的 y（1 号选 y 且 x 没选 y）」

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS (
  SELECT * FROM SC SCY
  WHERE SCY.Sno = '1'
  AND NOT EXISTS (
    SELECT * FROM SC SCZ
    WHERE SCZ.Sno = SCX.Sno AND SCZ.Cno = SCY.Cno
  )
);
```

套路二：「选修了全部课程」

命题转化：「对所有课程 x（x 被 y 选修）」等价于「不存在 x（y 没选修 x）」

```

SELECT Sname
FROM Student
WHERE NOT EXISTS (
  SELECT * FROM Course
  WHERE NOT EXISTS (
    SELECT * FROM SC
    WHERE Sno = Student.Sno AND Cno = Course.Cno
  )
);

```

金句：「全部 / 所有」量词 → SQL 用「双重 NOT EXISTS」实现。第一层 NOT EXISTS 否定「存在这样的课程」，第二层 NOT EXISTS 否定「学生没选这门课」。

七、集合查询 —— 把多个查询结果「拼起来」

操作	关键字	含义	是否去重
并	UNION / UNION ALL	两边都要	UNION 去重，UNION ALL 保留
交	INTERSECT	都要有	去重
差	EXCEPT	前有无后无	去重

硬约束：参与集合操作的查询列数必须相同，对应列数据类型必须相同。

```

-- CS 专业的学生 ∪ 年龄 ≤ 19 的学生
SELECT * FROM Student WHERE Sdept = 'CS'
UNION
SELECT * FROM Student WHERE Sage <= 19;

-- 既选 1 又选 2 的学生（交）
SELECT Sno FROM SC WHERE Cno = '1'
INTERSECT
SELECT Sno FROM SC WHERE Cno = '2';

```

八、基于派生表的查询 —— 把子查询「放」到 FROM 里

当子查询出现在 FROM 子句时，它变成一张临时的派生表，必须有别名。

```

-- 找出每个学生超过自己平均成绩的课程号
SELECT Sno, Cno
FROM SC,
  (SELECT Sno, AVG(Grade) AS avg_grade
   FROM SC
   GROUP BY Sno) AS Avg_sc(avg_sno, avg_grade)
WHERE SC.Sno = Avg_sc.avg_sno
AND SC.Grade >= Avg_sc.avg_grade;

```

金句：FROM 里的子查询 = 临时计算一个小表，再和主表做连接。AS 别名可省，但别名本身不能省



九、数据更新（DML）——三板斧

INSERT

```
-- (1) 插入完整元组
INSERT INTO TAB1 (C1, C2, C3, C4) VALUES (1, 2, 3, '4');

-- (2) 省略列名 → 必须按建表顺序给所有列赋值
INSERT INTO TAB1 VALUES (1, 2, 3, '4');

-- (3) 部分赋值 → 未赋值列自动为 NULL（除非该列有 NOT NULL 约束）
INSERT INTO TAB1 (C1, C2, C3) VALUES (1, 2, 3);

-- (4) 插入子查询结果（批量）
INSERT INTO TAB2 (C1, avg_C2)
SELECT C1, AVG(C2) FROM TAB1 GROUP BY C1;
```

UPDATE

```
-- (1) 改某一行
UPDATE TAB1 SET C4 = '0' WHERE C1 = 1;

-- (2) 改多行（省略 WHERE = 改全表）
UPDATE TAB1 SET C3 = C3 + 1;

-- (3) 带子查询
UPDATE TAB1 SET C4 = '0'
WHERE C1 IN (SELECT C1 FROM TAB2 WHERE avg_C2 = 2);
```

DELETE

```
-- (1) 删某些行
DELETE FROM TAB1 WHERE C1 = 1;

-- (2) 删全表（表结构还在！）
DELETE FROM TAB1;

-- (3) 带子查询
DELETE FROM TAB1
WHERE C1 IN (SELECT C1 FROM TAB2 WHERE avg_C2 = 2);
```

金句：DELETE 删的是数据，不是表的定义。表的结构（列、约束、类型）躺在数据字典里，删完数据表还在。

十、视图 —— 「虚拟表」面具



视图 = 存储在数据字典里的查询语句。本身不存数据，查询时再动态消解为对基本表的操作。

视图消解（query rewrite）

消解 = 把对视图的查询改写为对基本表的等价查询。

```
-- 定义视图
CREATE VIEW V_TAB1 AS
  SELECT C1, C2, C3, C4 FROM TAB1 WHERE C1 = 1;

-- 用户查询视图
SELECT * FROM V_TAB1 WHERE C2 = 2;

-- DBMS 自动消解为（把视图定义子查询展开到 FROM）
SELECT C1, C2, C3, C4
FROM  TAB1
WHERE  C1 = 1      -- 来自视图定义
      AND  C2 = 2;  -- 来自用户查询
```

金句：视图消解 = 把面具摘下来看底下的脸。DBMS 把视图的子查询当「子面具」嵌进 FROM，再把用户条件 AND 上去。行列子集视图的消解一般无障碍；带表达式 / 分组 / 多表的视图，消解后可能无法更新（因为不知道改写回哪张基本表）。

```
-- 基础视图
CREATE VIEW V_TAB1 AS
  SELECT C1, C2, C3, C4 FROM TAB1 WHERE C1 = 1;

-- 加 WITH CHECK OPTION → 增删改必须满足 WHERE 条件
CREATE VIEW V_TAB2 AS
  SELECT C1, C2, C3, C4 FROM TAB1 WHERE C4 = '4'
  WITH CHECK OPTION;

-- 分组视图（带聚合函数 + GROUP BY）
CREATE VIEW V_TAB5 (C1, avg_C2) AS
  SELECT C1, AVG(C2) FROM TAB1 GROUP BY C1;

-- 带虚拟列（表达式）—— 也叫「带表达式的视图」
CREATE VIEW V_TAB4 (C1, new_C2) AS
  SELECT C1, 10 + C2 FROM TAB1;

-- 建立在视图上的视图
CREATE VIEW V_TAB3 AS
  SELECT C1, C2, C3 FROM V_TAB1 WHERE C2 = 2;

-- 删除视图（CASCADE 把由它导出的视图一起删）
DROP VIEW V_TAB1 CASCADE; -- V_TAB3 也被删
DROP VIEW V_TAB2;        -- 不带 CASCADE，只删 V_TAB2
```

视图的 4 个分类

类型	特征	能否更新
行列子集视图 ★★	单表 + 去掉部分行/列 + 保留主码	可
带表达式的视图	选了计算列	不可（虚拟列没地方存）



分组视图	带聚集函数 / GROUP BY	不可
由多表/视图导出的视图	涉及连接	一般不可

金句：行列子集视图几乎全能干；带表达式 / 分组 / 多表的视图，只能查不能改。考试问「这个视图能否 UPDATE」—— 先看是不是「行列子集」。

十一、空值 —— SQL 的「第三个逻辑值」

运算	结果
空值 $\pm$ 数字	空值
空值 $>$ 数字	UNKNOWN (不是 TRUE, 也不是 FALSE)
判断空值	IS NULL / IS NOT NULL (不能 = NULL)

金句：WHERE / HAVING 只挑条件为 TRUE 的元组。UNKNOWN 会被过滤掉——它既不是真，也不是假，它「悬而未决」。

易混淆对比总结

对比项	区别	一句话口诀
WHERE vs HAVING	行过滤 vs 组过滤；WHERE 不能接聚集函数	「WHERE 管行，HAVING 管组」
CASCADE vs RESTRICT	级联连带删 vs 拒绝删除	「CASCADE 砍树枝，RESTRICT 金钟罩」
= ANY vs IN	语义相同，可互换	「= ANY 就是 IN 的双胞胎」
!= ALL vs NOT IN	语义相同	「!= ALL = NOT IN」
视图 vs 基本表	视图不存数据，只存查询	「视图是面具，基本表是脸」
UNION vs UNION ALL	UNION 去重，UNION ALL 保留	「UNION 默认瘦身」
EXISTS vs IN	EXISTS 检子查询是否非空，IN 检元素	「EXISTS 看有没有，IN 看等不等」
NOT NULL 约束位置	只能列级	「NOT NULL 是单兵，不能组战队」
主码单/组合	单列主码可列级，组合主码必须表级	「组队的主码必表级」

逻辑主线



SQL (人话 ↔ 数据库的翻译官)

- |
- |—— 一、DDL: 盖房子 ——|—— 模式 SCHEMA ——|
 - |—— 基本表 TABLE ——|—— 都支持 CASCADE / RESTRICT
 - |—— 索引 INDEX ——|
- |
- |—— 二、SELECT: 问问题 (本章核心)
- | |
- | |—— 单表: WHERE 过滤 → 投影 (别名 / DISTINCT / 计算列)
- | |
- | |—— 聚集 + 分组: 聚集函数 → GROUP BY 切组 → HAVING 筛组
- | |
- | |—— 多表: 等值连接 / 自身连接 / 外连接 (LEFT/RIGHT/FULL)
- | |
- | |—— 嵌套查询: IN / 比较 / ANY-ALL / EXISTS (双重否定)
 - | |—— 不相关子查询 vs 相关子查询
- | |
- | |—— 集合查询: UNION / INTERSECT / EXCEPT (列数+类型要齐)
- | |
- | |—— 派生表: 子查询进 FROM → 必须起别名
- |
- |—— 三、DML: 改数据
- | |—— INSERT (单行 / 子查询批量)
- | |—— UPDATE (带子查询)
- | |—— DELETE (删数据不删结构)
- |
- |—— 四、视图: 面具下的查询
- | |—— 行列子集视图 (可改) / 分组视图 (不可改) / 表达式视图
- | |—— WITH CHECK OPTION / CASCADE 级联删
- |
- |—— 五、空值: 第三个逻辑值
- | |—— IS NULL / 算术空 / 比较 UNKNOWN / WHERE 过滤 UNKNOWN

结语金句: SQL 不难, 难的是在脑子里同时跑出「我写了什么」和「数据库实际先做什么」。一旦把执行顺序 (FROM→WHERE→GROUP→HAVING→SELECT→DISTINCT→ORDER) 刻进肌肉记忆, 所有嵌套、所有分组、所有 EXISTS 都能顺势拆开。

